



Curso: Compiladores e interpretes

Sprint 3: Proyecto Final

Profesor: Marco Hernandez Vasquez

Estudiantes:

Andres Blanco 2022108841

Giancarlo Vega Marín 2020195338

Sebastián Chaves 2021032506

Fecha de Entrega: 27/11/2025

Semestre II

Introduccion:

En este tercer sprint del proyecto LogoTec se realizó la integración del compilador con un robot tortuga físico controlado mediante un microcontrolador ESP32, un driver de motor L293D, 4 motores de corriente directa y un servomotor para el lápiz.

El objetivo principal fue lograr que el mismo flujo de compilación desarrollado en los sprints anteriores pudiera ejecutarse en modo simulación, usando el visor de tortuga en java y también ejecutarse en modo físico por medio del robot, enviando comandos al ESP32 a través de una conexión TCP/WIFI.

Diagrama de arquitectura:

Compilación y ejecución:

El flujo de uso del sistema Logo Tec después del Sprint 3 es el siguiente.

1. Compilación del proyecto Java.

Se utiliza Maven para compilar el proyecto. Desde la carpeta del proyecto se ejecuta el comando de compilación, lo cual genera las clases en el directorio target.

2. Compilación de un programa Logo Tec.

Se ejecuta el compilador sobre un archivo fuente con extensión smp, indicando la ruta del directorio de salida. Como resultado se obtienen varios archivos intermedios, entre ellos un archivo objeto con extensión lojb que contiene el programa en formato ejecutable para la máquina virtual.

3. Ejecución en modo simulación.

Si se ejecuta RuntimePlayer solo con la ruta del archivo lojb, el runtime lee el archivo objeto, lo interpreta en la máquina virtual y utiliza TurtleRuntime para generar los comandos de dibujo para el visor gráfico. En este modo no se establece conexión con el robot, por lo que los comandos de ArduinoController se quedan en el buffer.

4. Ejecución en modo robot físico.

Para usar el robot, primero se carga el firmware `esp32_servo_motor.ino` en el ESP32, se enciende el robot y se verifica en el monitor serie que se haya conectado a la red WiFi. Allí también se puede consultar la dirección IP asignada por el router. Luego se ejecuta `RuntimePlayer` indicando la ruta del archivo `lobj` y la IP del ESP32. El runtime crea una instancia de `TurtleRuntime`, intenta conectarse al ESP32 mediante `ArduinoController` y, si tiene éxito, envía el comando INICIO. A partir de ese momento, cada instrucción de tortuga se refleja tanto en el estado gráfico como en el robot físico. Al finalizar, se envía el comando FIN y se cierra la conexión.

En el proyecto existe además un script por lotes de Windows que automatiza estos pasos, permitiendo compilar un programa y ejecutarlo directamente en el robot con una IP preconfigurada.

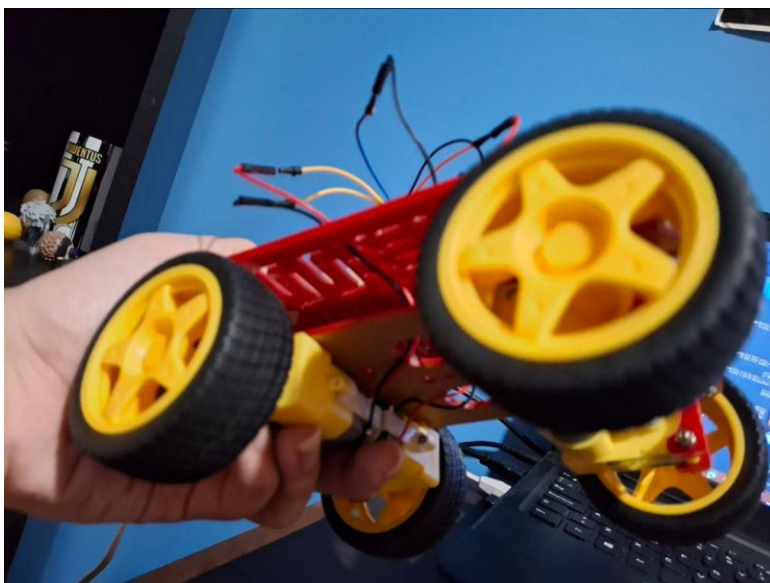
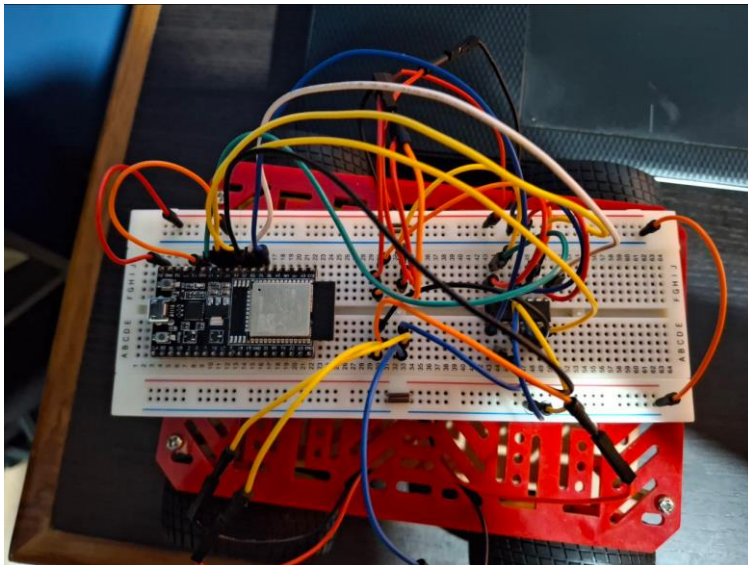
Se realizó una interfaz en Swing, partiendo del código de Java ya utilizado en los sprints anteriores.

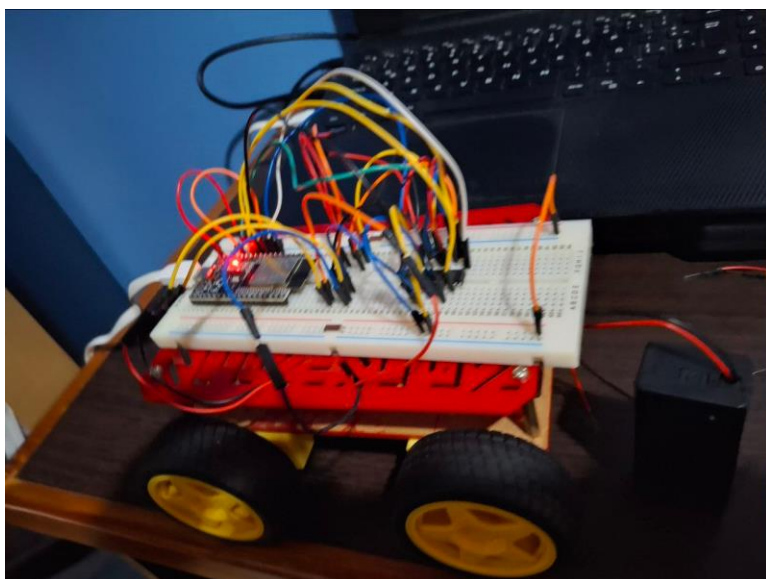


El objetivo de esta interfaz es realizar la compilación de los códigos de prueba, de una forma más eficiente, sin tener que escribir líneas de comando en la terminal. La interfaz permite subir código, guardar código nuevo basado en lo escrito en el cuadro de texto, y finalmente compilar dicho código seleccionado.

Se crearon dos archivos .bat, los cuales permiten la ejecución del archivo ejecutable, posterior a la compilación. De igual manera, permite al usuario una manera sencilla de ejecutar la función, sin necesidad de la terminal, y permitiendo múltiples ejecuciones seguidas. Un archivo genera el dibujo; el otro además del dibujo, se conecta al ESP32 para enviar las señales a la “tortuga”.

Posteriormente, se procedió a construir la “tortuga”, tomando el diseño del carro básico de un proyecto anterior realizado en la carrera.





Este dispositivo recibe las señales por medio del ESP32, enviando las señales del motor al driver L932D, permitiendo el movimiento de los motores, los cuales están alimentados por una pila de 9V.

De igual forma, se encuentra un servomotor SG90, el cual simula el movimiento del lápiz. Los comandos “bajalapiz” y “subelapiz” giran el servomotor, estableciendo la posición del lápiz pegado a uno de las cabezas del servomotor.

Problemas encontrados:

Durante el desarrollo del Sprint 3 se identificaron varios retos técnicos que se resumen a continuación.

1. Conexión WiFi y manejo de errores.

Al inicio se presentaron dificultades para establecer y mantener la conexión TCP entre la computadora y el ESP32, especialmente cuando el robot se encontraba en una red con más dispositivos o cuando se reiniciaba el microcontrolador. Para mejorar la robustez se agregaron mensajes de error claros en la parte de Java y se simplificó el protocolo de comandos a líneas de texto con formato fijo. En el lado del ESP32 se revisó el ciclo de aceptación de clientes y se mejoró el control del buffer de entrada.

2. Escala de movimientos y calibración.

En el lenguaje LogoTec se piensa en distancias y pasos lógicos, pero en el firmware actual, los valores enviados en los comandos AVANZAR y RETROCEDER se interpretan directamente como milisegundos de movimiento del motor. Esto provoca que la relación entre unidades del programa y centímetros recorridos dependa de factores físicos como la velocidad real de los motores, el voltaje de la batería y el rozamiento con la superficie. En este sprint se definieron valores aproximados que permiten visualizar las figuras.

3. Implementación de giros físicos.

Los comandos GIRAR_DERECHA y GIRAR_IZQUIERDA se manejan correctamente en el runtime lógico, pero su implementación física en el firmware aún necesita ajustes. Actualmente, los giros dependen de tiempos de activación de los motores que no han sido calibrados con precisión para producir rotaciones específicas como 90 grados.

4. Referencia física del comando CENTRO.
El comando CENTRO reposiciona a la tortuga en el origen del sistema de coordenadas lógicas dentro de la simulación, pero no existe un mecanismo automático para recolocar físicamente al robot en un punto fijo del área de dibujo. Esto requiere que el usuario ubique manualmente al robot antes de cada ejecución.

5. Cambios de color en hardware.
El comando COLOR se utiliza para cambiar el color del trazo en el visor gráfico y se envía como mensaje al ESP32. Sin embargo, en el hardware actual no existe un mecanismo para cambiar de lápiz o color físico, por lo que el color se mantiene como un atributo lógico.

Problemas conocidos:

Aunque el sistema es funcional y permite ejecutar programas de LogoTec en el robot tortuga, existen problemas conocidos que no se resolvieron completamente en este sprint:

1. Falta de calibración precisa de distancias.

La relación entre las unidades usadas en los programas Logo Tec y los centímetros recorridos por el robot aún no está calibrada con exactitud. Esto provoca que en figuras grandes se acumulen errores de posición.

2. Giros no estandarizados.

Los tiempos de giro para ángulos específicos como 90 o 45 grados no se han fijado de manera experimental, por lo que la tortuga física puede girar ligeramente más o menos de lo esperado.

3. Dependencia de la superficie de dibujo.

El comportamiento del robot varía según la fricción del piso o del papel donde se dibuja. En superficies muy lisas o ásperas se observan desviaciones mayores.

4. Ausencia de recolocación automática al centro.

El comando CENTRO no recoloca físicamente al robot en un punto de referencia. Esto obliga a que el usuario lo ubique manualmente al inicio de cada ejecución.

5. Cambio de color solo lógico.

No se implementó un mecanismo físico para cambiar de lápiz o color. El color de línea es solo un atributo visual en la simulación y en la documentación de las figuras.

Conclusiones y recomendaciones

Al finalizar el proyecto, se llegaron a distintas conclusiones. Se cumple el objetivo de implementar un compilador funcional y completo para el lenguaje de programación escogido. Se logran crear el Frontend y el Backend del compilador, así como una conexión exitosa entre ambos, que realice todo el proceso de compilación de cualquier código enviado, siguiendo las reglas de sintaxis y semánticas. Se logra la conexión entre el código objeto generado y un dispositivo de hardware, pero no se logra implementar completamente todas las funciones solicitadas en el dispositivo de hardware, a pesar de que el software si logra crear todo el dibujo solicitado. Gracias al proyecto, se logra apreciar más a fondo la importancia de los compiladores en los programas y lenguajes que se utilizan a diario como personas, y futuros ingenieros.

Entre las recomendaciones, se destaca la importancia de obtener los componentes de hardware con anticipación, ya que no siempre estarán disponibles en los lugares usuales de venta. A su vez, es recomendable tener múltiples componentes de reserva, en el caso de que uno no funcione como se debe, o falle durante el ensamblaje. Se recomienda también prestar mucha atención a la sintaxis y ortografía solicitada en la gramática, ya que un carácter faltante, adicional o diferente puede afectar gravemente a la hora de revisión o incluso compilación y ejecución. Por último, se recomienda investigar para futuro, el correcto funcionamiento de un carro inalámbrico, controlado por medio de motores DC y Arduino.

Referencias:

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). Compilers: Principles, techniques, and tools (2nd ed.). Addison Wesley. <https://www.pearson.com/en-us/subject-catalog/p/aho-compilers-principles-techniques-and-tools-2nd-edition/P200000003472>

Cooper, K. D., & Torczon, L. (2011). Engineering a compiler (2nd ed.). Morgan Kaufmann. <https://shop.elsevier.com/books/engineering-a-compiler/cooper/978-0-12-088478-0>

Espressif Systems. (n.d.). ESP32 documentation. Espressif Systems. <https://docs.espressif.com/>

Instituto Tecnológico de Costa Rica. (2025). Apuntes y material del curso CE-1108 Compiladores e Intérpretes, II semestre 2025.